

SWHarden.com

The personal website of Scott W Harden

Run Llama 2 Locally with Python

How to run open source large language models locally to provide AI chat responses to text prompts

July 29, 2023

python ai

This page describes how to interact with the Llama 2 large language model (LLM) locally using Python, without requiring internet, registration, or API keys. We will deliver prompts to the model and get AI-generated chat responses using the [llama-cpp-python](#) package.

UPDATE: A C# version of this article has been created. Code on this page describes a Python-centric strategy for running the Llama2 LLM locally, but a newer article I wrote describes [how to run AI chat locally using C#](#) (including how to have it answer questions about documents) which some users may find easier to follow.

Step 1: Download a Large Language Model

The Llama 2 model can be downloaded in [GGML format](#) from Hugging Face:

- Model I'm using: [llama-2-7b-chat.ggmlv3.q8_0.bin](#) (7 GB)
- All models: [Llama-2-7B-Chat-GGML/tree/main](#)
- Model descriptions: [Readme](#)

The model I'm using here is the largest and slowest one currently available. It is 7 GB in size and requires 10 GB of ram to run. Developers should experiment with different models, as simpler models may run faster and produce similar results for less complex tasks.

Step 2: Prepare the Python Environment

- Install the latest version of Python from [python.org](#)
- Create a virtual environment: `python -m venv .venv`
- Activate the virtual environment: `.venv/Scripts/activate`
- Install the [llama-cpp-python](#) package: `pip install llama-cpp-python`

Installation will fail if a C++ compiler cannot be located. To get one:

- **Windows:** Install [Visual Studio Community](#) with the "Desktop development with C++" workload. It is free for individuals an open-source developers. See the [C++ installation guide](#) for more information.
- **Linux:** `apt install python3-dev`
- **MacOS:** `brew install python3-dev`

Step 3: Interact with the Llama 2 large language model

Create a new python script and run it inside the virtual environment:

```
# Load the Large Language model file
from llama_cpp import Llama
LLM = Llama(model_path="./llama-2-7b-chat.ggmlv3.q8_0.bin")

# create a text prompt
prompt = "Q: What are the names of the days of the week? A:"

# generate a response (takes several seconds)
output = LLM(prompt)

# display the response
print(output["choices"][0]["text"])
```

It took my system 7 seconds to generate this response:

Question: *What are the names of the days of the week?*

Answer: *The names of the days of the week, in order, are: Monday Tuesday Wednesday Thursday Friday Saturday Sunday*

Generating Longer Responses

To get longer responses to complex questions, set `n_ctx` to a larger number to increase the number of tokens the large language model uses for context, then remove the `max_tokens` limit for the query. While this allows longer responses, it can significantly increase the total time required to generate a response.

```
from llama_cpp import Llama

# define n_ctx manually to permit larger contexts
LLM = Llama(model_path="./llama-2-7b-chat.ggmlv3.q8_0.bin", n_ctx=2048)

# create a text prompt
prompt = "Q: Why are Jupyter notebooks difficult to maintain? A:"

# set max_tokens to 0 to remove the response size limit
output = LLM(prompt, max_tokens=0)

# display the response
print(output["choices"][0]["text"])
```

It took my system 1.5 minutes to generate this response:

Question: Why are Jupyter notebooks difficult to maintain?

Response:

Jupyter notebooks can be challenging to maintain for several reasons:

1. Temporary file system: Jupyter notebooks reside on the user's local disk, which can make them unreliable and difficult to maintain over time.
2. Version control issues: As notebooks are updated frequently, it becomes challenging to keep track of changes and manage different versions of a notebook. This can lead to conflicts when working with others on the same project.
3. Lack of organization: Since Jupyter notebooks are created manually, it's easy to lose track of which files are included in the notebook and how they relate to each other. This makes it challenging to maintain a well-organized structure for large projects with many components.
4. Limited automation: There are limited tools available for automating tasks within Jupyter notebooks, making it more challenging to create reproducible builds or automate the testing process.

5. Security concerns: Since Jupyter notebooks contain sensitive data such as user credentials, personal information, and intellectual property, they must be protected against unauthorized access and breaches. This can further complicate maintenance efforts.
6. Interactive interfaces: Jupyter notebooks are designed for interactive use, which means that any changes to the code or visualizations will affect the interface in real-time. This can make it challenging to maintain a consistent appearance while working on a project.
7. Limited collaboration support: Jupyter notebooks are primarily designed as an individual tool, which limits their ability to accommodate multiple users and their contributions simultaneously. As a result, collaboration becomes more complicated with larger teams.
8. Lack of integration with other tools: Jupyter notebooks do not integrate well with other popular development tools like Git or IDEs (Integrated Development Environments). This can make it difficult to manage multiple projects, keep track of changes, and collaborate effectively with team members.
9. Limited scalability: As projects grow in size and complexity, Jupyter notebooks may become less efficient and more challenging to maintain. This is particularly true for large-scale data science or machine learning projects that require a lot of processing power and memory.
10. Lack of formalized workflows: Since Jupyter notebooks are primarily designed as an interactive environment, they don't provide standardized workflows or processes for managing projects. This can result in a less organized and more challenging maintenance experience for project teams.

By understanding these challenges, you can better prepare yourself to manage the complexities of Jupyter notebooks and develop strategies to address them effectively.

Using Llama 2 AI Chat in a Jupyter Notebook

Here is a standalone Jupyter notebook that demonstrates how to use different large language models to generate AI chat responses to plain text prompts. This notebook contains a few extra features to improve formatting of the output as well.

- **View Notebook:** [llama2-quickstart.html](https://github.com/Runeship/llama2-quickstart/blob/main/llama2-quickstart.html)
- **Download Notebook:** [llama2-quickstart.ipynb](https://github.com/Runeship/llama2-quickstart/blob/main/llama2-quickstart.ipynb)

Answering Questions about Local Documents

The AI-generated responses above only contain information built into the model itself. What if we want to give our language model custom information then have it be able to answer questions about it?

My next post [Using Llama 2 to Answer Questions About Local Documents](#) explores how to have the AI interpret information from local documents so it can answer questions about their content using AI chat.

Resources

- Meta: [Introducing Llama 2](#)
- Hugging Face: [Vigogne 2 13B Instruct - GGML](#)
- GitHub: [llama.cpp: Inference of LLaMA model in pure C/C++](#)
- PyPi: [llama-cpp-python](#) package
- ReadTheDocs: [llama-cpp-python package documentation](#)

[Feedback](#)[Source](#)[Contact](#)

Newer

[Using Llama 2 to Answer Questions About Local Documents](#)

July 30, 2023

Older

[Static Site Broken Link Detection](#)

May 13, 2023

Copyright © 2024 Scott W Harden

Built with Hugo 0.111.3 on June 12, 2024 at 0:16 am EST

<https://github.com/swharden/swharden.com>